BÀI THỰC HÀNH

LAB 11

Giám sát dữ liệu chuyển động với MPU6050

Đo gia tốc và vận tốc góc với MPU6050 qua EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Mục lục

1 Giới thiệu . . . 2
1.1 Tổng quan bài lab . . . 2
1.2 Mục tiêu . . . 2
2 Kiến thức nền . . . 2
2.1 Gia tốc kế và con quay hồi chuyển . . . 2
2.2 Giao tiếp I2C . . . 3
3 Thiết lập phần cứng . . . 3
3.1 Phần cứng sử dụng . . . 3
3.2 Ánh xạ tín hiệu I2C . . . 3
3.3 Kết nối MPU6050 . . . 3
4 EduFramework API . . . 4
4.1 Cấu trúc MPU_Data_t . . . 4
4.2 MPU_Begin() . . . 5
4.3 MPU_ReadData() . . . 5
5 Bài thực hành . . . 6
5.1 Yêu cầu . . . 6
5.2 Chương trình . . . 7
5.3 Giải thích chương trình . . . 8
5.4 Kiểm chứng . . . 8
6 Mở rộng . . . 8
6.1 Đọc chọn lọc và kiểm tra trạng thái . . . 8
6.2 Hiệu chuẩn với MPU_Calibrate() . . . 9
6.3 Địa chỉ I2C và chân AD0 . . . 9
6.4 Tùy chỉnh dải đo . . . 10
6.5 API nâng cao theo ngữ cảnh . . . 10
6.6 Bài tập mở rộng . . . 11
7 Tài liệu tham khảo . . . 11

1 Giới thiệu

1.1 Tổng quan bài lab

MPU6050 là cảm biến chuyển động sáu trục, tích hợp gia tốc kế ba trục và con quay hồi chuyển ba trục. Thiết bị đo gia tốc riêng và vận tốc góc theo các trục X, Y, Z, nhờ đó cung cấp dữ liệu về trạng thái chuyển động của vật thể gắn cảm biến [3] .

Bài lab sử dụng giao diện lập trình thiết bị (Device API) của EduFramework thay vì thao tác trực tiếp với thanh ghi MPU6050. Chương trình khởi tạo cảm biến, đọc gia tốc, vận tốc góc và nhiệt độ bên trong cảm biến, sau đó hiển thị các giá trị trên Serial Monitor. Các chức năng đọc riêng từng đại lượng, hiệu chuẩn, chọn địa chỉ I2C và dải đo được giới thiệu ở phần Mở rộng [6] .

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả vai trò của gia tốc kế và con quay hồi chuyển trong một cảm biến chuyển động sáu trục.
2. Giải thích các thành phần cơ bản của giao tiếp I2C gồm `SDA` , `SCL` và địa chỉ thiết bị.
3. Sử dụng cấu trúc `MPU_Data_t` để truy cập dữ liệu gia tốc, vận tốc góc và nhiệt độ cảm biến do EduFramework cung cấp.
4. Sử dụng `MPU_Begin()` và `MPU_ReadData()` để xây dựng ứng dụng giám sát dữ liệu MPU6050 trên Serial Monitor.
5. Nhận biết các khả năng cấu hình nâng cao của MPU6050 Device API, bao gồm hiệu chuẩn, thay đổi địa chỉ I2C và lựa chọn dải đo.

2 Kiến thức nền

2.1 Gia tốc kế và con quay hồi chuyển

MPU6050 là một đơn vị đo quán tính (Inertial Measurement Unit - IMU) gồm gia tốc kế và con quay hồi chuyển ba trục. Gia tốc kế đo **gia tốc riêng** (specific force), còn con quay hồi chuyển đo vận tốc góc quanh từng trục. Hai loại dữ liệu này bổ sung cho nhau khi quan sát chuyển động của cảm biến [1] [3] .

Trong EduFramework, dữ liệu gia tốc được biểu diễn bằng đơn vị `g` (gia tốc trọng trường tiêu chuẩn), còn vận tốc góc sử dụng `dps` (độ trên giây). Khi cảm biến đứng yên, gia tốc kế vẫn đo ảnh hưởng của trọng lực: nếu một trục gần thẳng đứng, giá trị trên trục đó có độ lớn xấp xỉ `1 g` . Ngược lại, các giá trị vận tốc góc thường gần `0 dps` , nhưng vẫn có thể xuất hiện sai lệch điểm không (offset) và nhiễu [1] [6] .

Ghi chú

Con quay hồi chuyển đo vận tốc góc, không trực tiếp trả về góc quay. Việc ước lượng các góc định hướng như góc nghiêng ngang (roll), nghiêng dọc (pitch) hoặc hướng quay (yaw) cần xử lý dữ liệu bổ sung và không thuộc phạm vi bài lab này.

2.2 Giao tiếp I2C

I2C là bus truyền thông nối tiếp đồng bộ sử dụng hai đường tín hiệu: `SCL` (Serial Clock) truyền xung nhịp và `SDA` (Serial Data) truyền dữ liệu. Nhiều thiết bị có thể dùng chung hai đường tín hiệu; bộ điều khiển phân biệt từng thiết bị thông qua địa chỉ I2C [2] .

Trong bài lab, S32K144 giao tiếp với MPU6050 qua I2C. Chân `AD0` quyết định bit thấp nhất của địa chỉ I2C 7 bit: ở mức logic thấp, địa chỉ là `0x68` ; ở mức logic cao, địa chỉ là `0x69` [3] .

Luồng dữ liệu của ứng dụng:

`MPU6050` → `I2C` → `Wire / LPI2C` → `MPU6050 Device API` → `Ứng dụng`

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài thực hành sử dụng MaaZEDU Development Board và một module MPU6050. MaaZEDU là bo mạch phát triển sử dụng vi điều khiển NXP S32K144 [5] [4] . Module được cấp nguồn `3.3V` , nối chung `GND` với bo mạch và trao đổi dữ liệu qua giao tiếp I2C [3] .

Phần cứng	Mô tả
MaaZEDU Development Board	Bo mạch phát triển sử dụng vi điều khiển S32K144.
MPU6050 module	Module cảm biến tích hợp gia tốc kế ba trục và con quay hồi chuyển ba trục.
Dây nối	Kết nối nguồn và các tín hiệu I2C giữa MPU6050 với MaaZEDU.
Cáp USB	Kết nối board với máy tính để cấp nguồn, nạp chương trình và sử dụng Serial Monitor.

Bảng 11.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ tín hiệu I2C

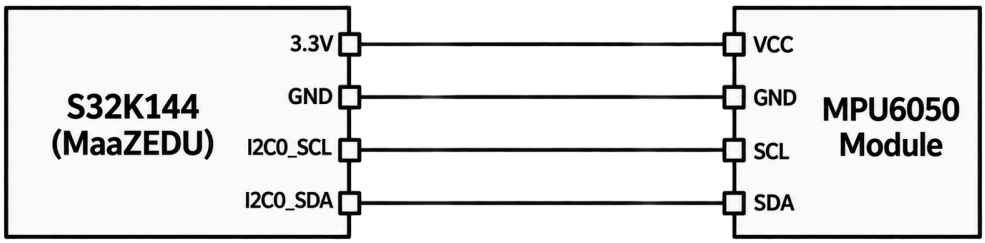
Trong cấu hình EduFramework sử dụng cho bài lab, bus I2C0 được ánh xạ tới `I2C0_SCL` (`PTA3`) và `I2C0_SDA` (`PTA2`). Giao diện Wire và trình điều khiển MPU6050 dùng các chân này để trao đổi dữ liệu [6] .

Thành phần	Logical Pin	MCU Pin	Chức năng
MPU6050 `SCL`	`I2C0_SCL`	`PTA3`	Xung nhịp I2C
MPU6050 `SDA`	`I2C0_SDA`	`PTA2`	Dữ liệu I2C

Bảng 11.3.2. Ánh xạ tín hiệu MPU6050 sử dụng trong bài thực hành

3.3 Kết nối MPU6050

Nối `VCC` của module với `3.3V` , `GND` với `GND` , `SCL` với `I2C0_SCL` và `SDA` với `I2C0_SDA` . Để dùng địa chỉ mặc định `0x68` , chân `AD0` phải ở mức logic thấp. Nếu module không có sẵn mạch kéo xuống, cần nối `AD0` với `GND` ; kiểm tra sơ đồ chân của module thực tế trước khi cấp nguồn [3] [6] .



Hình 11.3.1. Bốn kết nối nguồn và I2C giữa MPU6050 với MaaZEDU

Ghi chú

Hình minh họa bốn kết nối chính và không thể hiện chân `AD0`. Trước khi chạy chương trình, cần bảo đảm `AD0` ở mức thấp để sử dụng địa chỉ `0x68`. Chân `INT` không được sử dụng vì chương trình đọc dữ liệu theo chu kỳ bằng `MPU_ReadData()`.

4 EduFramework API

EduFramework cung cấp nhóm API cơ bản (Beginner API) để sử dụng MPU6050 với cấu hình mặc định và nhóm API nâng cao (Advanced API) để quản lý cấu hình thông qua một đối tượng ngữ cảnh. Bài thực hành chính chỉ cần `MPU_Data_t`, `MPU_Begin()` và `MPU_ReadData()`. Các API khác được giới thiệu trong phần Mở rộng [6].

4.1 Cấu trúc MPU_Data_t

`MPU_Data_t` là cấu trúc lưu kết quả đo trên ba trục và nhiệt độ cảm biến. Sau khi `MPU_ReadData()` trả về `true`, ứng dụng truy cập từng thành phần của biến dữ liệu bằng toán tử dấu chấm (`.`) [6].

```
typedef struct
{
    float accelX;
    float accelY;
    float accelZ;
    float gyroX;
    float gyroY;
    float gyroZ;
    float temperature;
} MPU_Data_t;
```

Mã nguồn 11.4.1. Cấu trúc dữ liệu `MPU_Data_t`

Trường	Đại lượng	Đơn vị
`accelX`	Gia tốc riêng theo trục X	`g`
`accelY`	Gia tốc riêng theo trục Y	`g`
`accelZ`	Gia tốc riêng theo trục Z	`g`
`gyroX`	Vận tốc góc theo trục X	`dps`
`gyroY`	Vận tốc góc theo trục Y	`dps`
`gyroZ`	Vận tốc góc theo trục Z	`dps`
`temperature`	Nhiệt độ cảm biến bên trong MPU6050	`°C`

Bảng 11.4.1. Các trường dữ liệu của `MPU_Data_t`

Ghi chú

Trường `temperature` là nhiệt độ đo tại cảm biến bên trong MPU6050, không phải phép đo trực tiếp nhiệt độ không khí xung quanh [3] .

4.2 MPU_Begin()

`MPU_Begin()` khởi tạo MPU6050 theo cấu hình mặc định của EduFramework. Theo giao diện được mô tả trong mã nguồn dự án, API khởi tạo tầng Wire, sử dụng địa chỉ `0x68`, đánh thức cảm biến và chọn dải đo `±2 g` cho gia tốc kế, `±250 dps` cho con quay hồi chuyển [6] .

`MPU_Begin`

Khởi tạo MPU6050 bằng cấu hình mặc định của EduFramework.

Cú pháp

```
MPU_Begin();
```

Giá trị trả về

`true` khi khởi tạo và kiểm tra giao tiếp thành công; `false` khi không thể giao tiếp hoặc cấu hình cảm biến.

Ví dụ:

```
if (true == MPU_Begin())
{
    /* MPU6050 is ready. */
}
```

4.3 MPU_ReadData()

`MPU_ReadData()` đọc một khung dữ liệu đo từ MPU6050 và cập nhật các trường gia tốc, vận tốc góc và nhiệt độ của `MPU_Data_t`. Hàm chỉ trả kết quả thành công khi cảm biến đã được khởi tạo và quá trình đọc không gặp lỗi [6] .

MPU_ReadData

Đọc khung dữ liệu đo và cập nhật biến dữ liệu được truyền vào.

Cú pháp

```
MPU_ReadData(data);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
data	Địa chỉ vùng lưu dữ liệu	Địa chỉ của biến `MPU_Data_t` nhận kết quả đo, ví dụ `&data`.

Giá trị trả về

`true` nếu đọc thành công; `false` nếu cảm biến chưa được khởi tạo, tham số không hợp lệ hoặc giao tiếp I2C gặp lỗi.
Ví dụ:

```
MPU_Data_t data;

if (true == MPU_ReadData(&data))
{
    Serial1_printFloat(data.accelX);
}
```

5 Bài thực hành

5.1 Yêu cầu

Viết chương trình đọc gia tốc, vận tốc góc trên ba trục và nhiệt độ bên trong MPU6050, sau đó hiển thị kết quả lên Serial Monitor. Cảm biến được khởi tạo bằng `MPU_Begin()`; trong vòng lặp, chương trình gọi `MPU_ReadData()` và chờ `1000 ms` trước chu kỳ tiếp theo.
Serial Monitor sử dụng tốc độ `9600 baud`. Nếu khởi tạo thất bại, chương trình in thông báo lỗi và không tiếp tục đọc. Nếu một lần đọc thất bại, chương trình báo lỗi rồi thử lại ở vòng lặp kế tiếp.

5.2 Chương trình

Trong `src/main.c`, sử dụng chương trình đã kiểm thử với phần cứng:

```
#include "Arduino.h"
#include "MPU6050.h"

int main(void)
{
    MPU_Data_t data;
    setup();
    Serial1_begin(9600U);
    Serial1_println("MPU6050 Sensor Monitor");
    Serial1_println("----------------------");
    if (false == MPU_Begin())
    {
        Serial1_println("MPU6050 initialization failed.");
        while (1)
        {
            delay(1000U);
        }
    }
    Serial1_println("MPU6050 initialized.");
    while (1)
    {
        if (true == MPU_ReadData(&data))
        {
            Serial1_print("Accel X: ");
            Serial1_printFloat(data.accelX);
            Serial1_print(" g | Y: ");
            Serial1_printFloat(data.accelY);
            Serial1_print(" g | Z: ");
            Serial1_printFloat(data.accelZ);
            Serial1_println(" g");
            Serial1_print("Gyro X: ");
            Serial1_printFloat(data.gyroX);
            Serial1_print(" dps | Y: ");
            Serial1_printFloat(data.gyroY);
            Serial1_print(" dps | Z: ");
            Serial1_printFloat(data.gyroZ);
            Serial1_println(" dps");
            Serial1_print("Temperature: ");
            Serial1_printFloat(data.temperature);
            Serial1_println(" C");
            Serial1_println("----------------------");
        }
        else
        {
            Serial1_println("MPU6050 read failed.");
        }
        delay(1000U);
    }
}
```

Mã nguồn 11.5.1. Chương trình giám sát dữ liệu MPU6050

5.3 Giải thích chương trình

`setup()` khởi tạo các thành phần nền tảng của EduFramework; `Serial1_begin(9600U)` cấu hình cổng Serial Monitor ở `9600 baud`. Tiếp theo, `MPU_Begin()` khởi tạo MPU6050. Nếu hàm trả về `false`, chương trình thông báo lỗi và không tiếp tục đọc cảm biến [6].
Mỗi vòng lặp gọi `MPU_ReadData(&data)` để nhận kết quả vào cấu trúc `data`. Khi hàm trả về `true`, chương trình in các thành phần `accelX`, `accelY`, `accelZ`, `gyroX`, `gyroY`, `gyroZ` và `temperature` qua các API Serial. Nếu đọc thất bại, chương trình in `MPU6050 read failed.` và thử lại ở vòng sau [6].
Luồng xử lý của ứng dụng có thể tóm tắt:
`Chuyển động` → `MPU6050` → `I2C` → `MPU_ReadData()` → `Serial1` → `Serial Monitor`
`delay(1000U)` tạo khoảng nghỉ một giây sau mỗi lần xử lý; chu kỳ cập nhật thực tế còn bao gồm thời gian đọc cảm biến và xuất dữ liệu Serial.

5.4 Kiểm chứng

Biên dịch và nạp chương trình xuống MaaZEDU, sau đó mở Serial Monitor với tốc độ `9600 baud`.
Trước tiên, đặt module yên trên một bề mặt ổn định. Khi một trục cảm biến gần thẳng theo phương trọng lực, thành phần gia tốc trên trục đó có độ lớn gần `1 g`, trong khi vận tốc góc trên các trục gần `0 dps`. Các giá trị có thể chênh lệch do hướng đặt module, độ lệch điểm không và nhiễu cảm biến [1].
Tiếp theo, nghiêng module để quan sát gia tốc trên từng trục, rồi xoay module để quan sát sự thay đổi của vận tốc góc. Không yêu cầu các giá trị trùng với một bộ số cố định; mục tiêu là kiểm chứng dữ liệu phản ứng hợp lý theo chuyển động thực tế.
Dữ liệu đã quan sát trong quá trình kiểm thử phần cứng có dạng:

```
MPU6050 Sensor Monitor
----------------------
MPU6050 initialized.
Accel X: 0.060 g | Y: 0.005 g | Z: 1.030 g
Gyro X: 1.901 dps | Y: -1.534 dps | Z: -0.191 dps
Temperature: 46.412 C
----------------------
```

KẾT QUẢ MONG ĐỢI

Serial Monitor hiển thị gia tốc, vận tốc góc theo ba trục và nhiệt độ bên trong MPU6050. Khi nghiêng hoặc xoay module, các giá trị đo thay đổi theo chuyển động; khi module đứng yên, vận tốc góc thường gần `0 dps` và gia tốc kế thể hiện ảnh hưởng của trọng lực theo hướng đặt cảm biến.

6 Mở rộng

Bài thực hành chính sử dụng API cơ bản (Beginner API) với cấu hình mặc định. Phần này giới thiệu các API đọc chọn lọc, hiệu chuẩn và API nâng cao (Advanced API) để lựa chọn địa chỉ I2C, dải đo và số mẫu hiệu chuẩn [6].

6.1 Đọc chọn lọc và kiểm tra trạng thái

Ngoài `MPU_ReadData()`, API cơ bản cung cấp những hàm đọc riêng từng nhóm dữ liệu và hàm kiểm tra trạng thái khởi tạo:

API	Chức năng
`MPU_IsInitialized()`	Kiểm tra trạng thái khởi tạo của thiết bị mặc định.
`MPU_ReadAcceleration()`	Đọc gia tốc theo ba trục X, Y, Z.
`MPU_ReadGyroscope()`	Đọc con quay hồi chuyển theo ba trục X, Y, Z.
`MPU_ReadTemperature()`	Đọc nhiệt độ cảm biến bên trong MPU6050.

Bảng 11.6.1. Các API đọc chọn lọc và kiểm tra trạng thái

Các hàm `MPU_ReadAcceleration()`, `MPU_ReadGyroscope()` và `MPU_ReadTemperature()` đọc riêng dữ liệu tương ứng từ cảm biến. Nếu cần cả ba nhóm dữ liệu, `MPU_ReadData()` phù hợp hơn vì cập nhật chúng trong một lần gọi API [6].

Ví dụ khi chỉ cần gia tốc:

```
float x = 0.0F;
float y = 0.0F;
float z = 0.0F;

if (true == MPU_ReadAcceleration(&x, &y, &z))
{
    /* Xu ly du lieu gia toc. */
}
```

6.2 Hiệu chuẩn với MPU_Calibrate()

`MPU_Calibrate()` hiệu chuẩn thiết bị mặc định bằng cách thu thập các mẫu đo khi cảm biến đứng yên, sau đó xác định độ lệch điểm không cho gia tốc kế và con quay hồi chuyển. Theo triển khai được mô tả trong nguồn EduFramework, API cơ bản sử dụng `500` mẫu và giả định cảm biến nằm ổn định với trục `+Z` chịu thành phần gia tốc khoảng `+1 g` [6].

`MPU_Calibrate`

Hiệu chuẩn MPU6050 mặc định và cập nhật độ lệch dùng cho các lần đọc sau.

Cú pháp

```
MPU_Calibrate();
```

Giá trị trả về

`true` nếu hiệu chuẩn hoàn thành; `false` nếu cảm biến chưa được khởi tạo hoặc gặp lỗi trong quá trình thu thập mẫu.

Ghi chú

Giữ module đứng yên trong suốt quá trình hiệu chuẩn và đặt theo hướng yêu cầu của thuật toán. Di chuyển cảm biến khi lấy mẫu sẽ làm kết quả hiệu chuẩn không phản ánh đúng trạng thái tĩnh.

6.3 Địa chỉ I2C và chân AD0

Khi `AD0` ở mức thấp, MPU6050 có địa chỉ I2C 7 bit là `0x68`; khi chân này ở mức cao, địa chỉ chuyển thành `0x69`. Vì vậy, có thể phân biệt hai MPU6050 trên cùng một bus I2C nếu mỗi thiết bị dùng một mức `AD0` khác nhau [3].

EduFramework định nghĩa hai hằng địa chỉ:

```
MPU6050_ADDRESS_DEFAULT   /* 0x68 */
MPU6050_ADDRESS_ALT       /* 0x69 */
```

API cơ bản sử dụng địa chỉ mặc định `0x68`. Để chọn địa chỉ khác, ứng dụng sử dụng API nâng cao với một biến ngữ cảnh `MPU6050_t`; tầng Wire phải được khởi tạo trước khi gọi `MPU6050_begin()` [6].
Ví dụ nguyên tắc:

```
MPU6050_t mpu;
Wire_begin();
if (true == MPU6050_begin(&mpu, MPU6050_ADDRESS_ALT))
{
    /* Device at address 0x69 is ready. */
}
```

Mã nguồn 11.6.1. Khởi tạo MPU6050 bằng địa chỉ I2C tùy chọn

6.4 Tùy chỉnh dải đo

Tài liệu kỹ thuật MPU6050 quy định bốn dải đo toàn thang (full-scale range) cho gia tốc kế và bốn dải cho con quay hồi chuyển. EduFramework cho phép thiết lập các dải này bằng API nâng cao [3] [6].

Gia tốc kế	Con quay hồi chuyển
`MPU6050_RANGE_2_G`	`MPU6050_RANGE_250_DEG`
`MPU6050_RANGE_4_G`	`MPU6050_RANGE_500_DEG`
`MPU6050_RANGE_8_G`	`MPU6050_RANGE_1000_DEG`
`MPU6050_RANGE_16_G`	`MPU6050_RANGE_2000_DEG`

Bảng 11.6.2. Các lựa chọn dải đo của MPU6050

Hai API cấu hình tương ứng:

```
MPU6050_setAccelerometerRange(&mpu, MPU6050_RANGE_4_G);
MPU6050_setGyroRange(&mpu, MPU6050_RANGE_500_DEG);
```

Dải đo toàn thang lớn hơn cho phép đo chuyển động có biên độ lớn hơn, nhưng hệ số nhạy (số đếm trên mỗi đơn vị đo) cũng thay đổi. Vì thế cần chọn dải đo theo mức gia tốc hoặc vận tốc góc dự kiến thay vì mặc định chọn dải lớn nhất [3].

6.5 API nâng cao theo ngữ cảnh

API nâng cao sử dụng biến ngữ cảnh `MPU6050_t` để quản lý địa chỉ I2C, dải đo, dữ liệu cảm biến và độ lệch hiệu chuẩn thay cho cấu hình mặc định [6].
Các API khởi tạo, cấu hình và thu thập dữ liệu được tổng hợp dưới đây:

Advanced API	Vai trò
`MPU6050_begin()`	Khởi tạo ngữ cảnh với địa chỉ I2C đã chọn.
`MPU6050_setAccelerometerRange()`	Thay đổi dải đo gia tốc kế.
`MPU6050_setGyroRange()`	Thay đổi dải đo con quay hồi chuyển.
`MPU6050_read()`	Đọc một khung dữ liệu đo vào ngữ cảnh.
`MPU6050_calibrate()`	Hiệu chuẩn với số mẫu do ứng dụng lựa chọn.

Bảng 11.6.3. API khởi tạo, cấu hình và đọc dữ liệu

Các hàm truy xuất dữ liệu đã lưu trong ngữ cảnh gồm:

Advanced API	Vai trò
`MPU6050_getAccelerationX()`, `MPU6050_getAccelerationY()`, `MPU6050_getAccelerationZ()`	Lấy giá trị gia tốc từng trục đã lưu trong ngữ cảnh.
`MPU6050_getGyroX()`, `MPU6050_getGyroY()`, `MPU6050_getGyroZ()`	Lấy giá trị vận tốc góc từng trục đã lưu trong ngữ cảnh.
`MPU6050_getTemperature()`	Lấy nhiệt độ đã lưu trong ngữ cảnh.

Bảng 11.6.4. API truy xuất dữ liệu từ ngữ cảnh

API cơ bản phù hợp với một cảm biến ở cấu hình mặc định. API nâng cao hỗ trợ địa chỉ I2C, dải đo, số mẫu hiệu chuẩn tùy chọn và các ngữ cảnh thiết bị riêng.

6.6 Bài tập mở rộng

Đặt `AD0` ở mức logic cao để dùng địa chỉ `0x69`, sau đó sử dụng API nâng cao để:

- Khởi tạo một `MPU6050_t` riêng;
- Chọn dải đo gia tốc kế `±4 g`;
- Chọn dải đo vận tốc góc `±500 dps`;
- Thực hiện hiệu chuẩn với số mẫu do chương trình lựa chọn;
- Đọc dữ liệu và hiển thị gia tốc cùng con quay hồi chuyển trên Serial Monitor.

Không gọi `MPU_Begin()` trong bài tập này. Sử dụng `MPU6050_begin()`, các hàm cấu hình dải đo và `MPU6050_calibrate()` theo API nâng cao.

KẾT QUẢ MONG ĐỢI

Cảm biến khởi tạo được tại địa chỉ `0x69` và trả dữ liệu sau khi thiết lập dải đo. Khi hiệu chuẩn đúng điều kiện tĩnh, độ lệch điểm không của các giá trị vận tốc góc có thể được giảm bớt.

7 Tài liệu tham khảo

[1] University of California, Berkeley, *Strapdown inertial navigation* , Rotations.

[2] Cornell University, *Inter-Integrated Circuit (I2C)* , ECE 4760 - Designing with Microcontrollers.

[3] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification* , PS-MPU-6000A-00, Rev. 3.4, 2013.

[4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.

[5] FPT Software, *MaaZ Edu Development Board - User Manual* , Rev. 1.0, 2025.

[6] EduFramework, *MPU6050 Device API* , EduFramework Source Code.